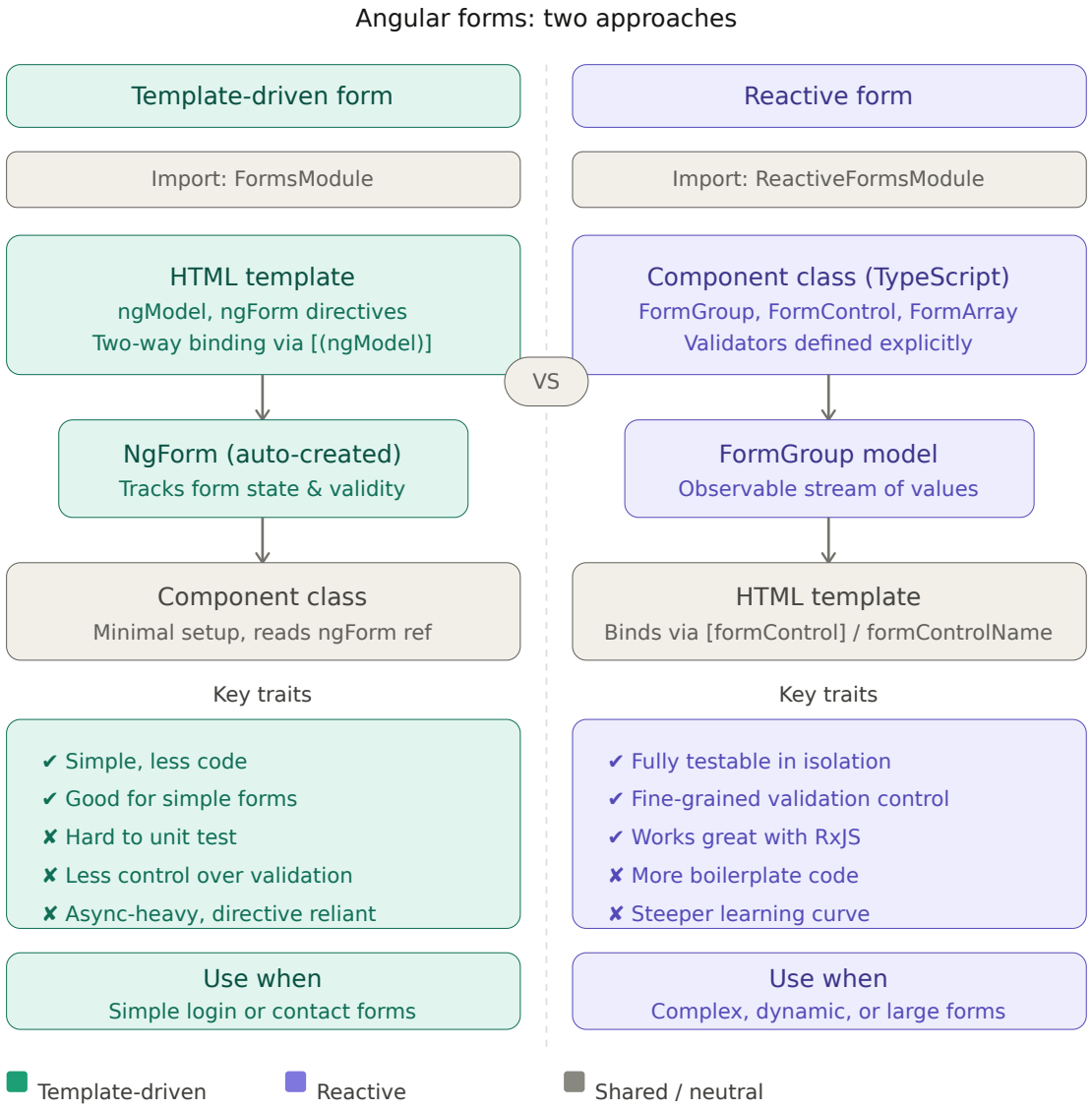
📘 Class 9 — Reactive Forms

Table of Contents

1. Template Driven vs Reactive Forms



2. Adding Reactive Forms to a Project

Step 1 — Import in component:

```
@Component({
  imports: [ReactiveFormsModule]  // required
})
```

Step 2 — Wire up the template:

```
<form [formGroup]="form" (ngSubmit)="saveBook()">
  <input formControlName="name"/>
  <button type="submit">Save</button>
</form>
```

Key directives:

Directive	Purpose
`[formGroup]="form"`	connects template form to TypeScript FormGroup
`formControlName="name"`	connects input to a specific control in FormGroup
`(ngSubmit)="saveBook()"`	calls method on form submit
`formGroupName="publisher"`	connects nested section to nested FormGroup

No `name` attribute needed:

```
<!-- Template Driven — name required -->
<input name="author" ngModel>

<!-- Reactive Forms — formControlName replaces name -->
<input formControlName="author">
```

---

3. Creating Form Controls — All Ways

Way 1 — Just initial value:

```
new FormControl('Asif')
// type: FormControl<string | null>
```

Way 2 — Initial value + single validator:

```
new FormControl('', Validators.required)
// type: FormControl<string | null>
```

Way 3 — Initial value + multiple validators:

```
new FormControl('', [Validators.required, Validators.minLength(3)])
// type: FormControl<string | null>
```

Way 4 — Object syntax with nonNullable only:

```
new FormControl('', { nonNullable: true })
// type: FormControl<string>  ← no null
```

Way 5 — Object syntax with nonNullable + single validator:

```
new FormControl('', { nonNullable: true, validators: Validators.required })
// type: FormControl<string>
```

Way 6 — Object syntax with nonNullable + multiple validators:

```
new FormControl('', {
  nonNullable: true,
  validators: [Validators.required, Validators.minLength(3)]
})
// type: FormControl<string>
```

Way 7 — Disabled at initialization:

```
new FormControl({ value: 'Asif', disabled: true }, { nonNullable: true })
// 1st argument: { value, disabled } object
// 2nd argument: options object
```

---

4. The Magic of nonNullable

Why FormControl always includes `null`:

When you call `.reset()` on a FormControl, Angular sets the value to `null` by default — not `''`, not `undefined`, but `null`.

```
form.controls.name.setValue('Asif')  // value = 'Asif'
form.controls.name.reset()           // value = null  ← Ang
ular default
```

TypeScript reflects this reality:

```
new FormControl('Asif')
// type: FormControl<string | null>
//                          ^^^^ because reset() can make i
t null
```

What `nonNullable: true` changes:

```
new FormControl('Asif', { nonNullable: true })
// reset() now returns to initial value 'Asif' — not null
// type: FormControl<string>  ← null removed
```

Comparison:

	After `reset()`	TypeScript type
Default	`null`	`string | null`
`nonNullable: true`	initial value (`'Asif'`)	`string`

> `nonNullable` literally means “this control can never be null.”

---

5. form.value vs form.getRawValue()

The type difference:

```
form = new FormGroup({
  name: new FormControl('Asif'),   // FormControl<string |
null>
  author: new FormControl('Author') // FormControl<string |
```

```
null>
})
```

Key difference — disabled controls:

- `form.value` — **excludes** disabled controls
- `form.getRawValue()` — **includes** disabled controls

```
// name control is disabled
form.value         // { author: 'Author' }              ← n
ame missing
form.getRawValue() // { name: 'Asif', author: 'Author' } ←
name included
```

---

6. Creating FormGroup

```
form = this.formBuilder.nonNullable.group({
  name: [''],
  author: ['']
})
```

Accessing controls — three ways:

```
this.form.controls['name']     // bracket — works on untype
d
this.form.get('name')          // method — most common in r
eal code, returns null-safe
this.form.controls.name        // dot — only works on typed
FormGroup
```

---

7. Simplifying with FormBuilder

Why FormBuilder exists:

Raw `FormGroup` requires `nonNullable: true` on every single control:

```
// Raw — repetitive
form = new FormGroup({
  name: new FormControl('', { nonNullable: true }),
  author: new FormControl('', { nonNullable: true }),
  category: new FormControl('', { nonNullable: true }),
  // repeat for every control...
})
```

FormBuilder applies `nonNullable` to all controls at once:

```
// FormBuilder — clean
form = this.formBuilder.nonNullable.group({
  name: ['', Validators.required],
  author: ['', Validators.required],
  category: ['']
})
```

Array syntax in FormBuilder:

```
['initialValue',  validators,          asyncValidators]
//  index 0        index 1              index 2
```

```
name: ['']                                          // valu
e only
name: ['', Validators.required]                     // + si
ngle validator
name: ['', [Validators.required, Validators.minLength(3)]]
// + multiple
```

---

8. inject() vs Constructor Injection

The problem with constructor injection at property level:

```
// ❌ error — formBuilder not ready when form is initialize
d
```

```
form = this.formBuilder.group({})  // runs at step 2 (properties)
constructor(private formBuilder: FormBuilder) {}  // runs at step 3
```

Execution order when `new MyClass()` is called:

```
1. Memory allocated
2. Property initializers run — top to bottom
3. Constructor body runs
4. Object ready
```

inject() fixes the order:

```
private formBuilder = inject(FormBuilder)  // step 2 — line 1
form = this.formBuilder.group({})          // step 2 — line 2 ✅ ready
```

Modern Angular pattern — no constructor needed:

```
export class EditBookReactive {
  private route = inject(ActivatedRoute)
  private bookService = inject(BooksService)
  private formBuilder = inject(FormBuilder)

  form = this.formBuilder.nonNullable.group({
    name: ['', Validators.required],
    author: ['', Validators.required]
  })
  // no constructor needed ✅
}
```

---

9. Disabling Controls

Wrong way — HTML attribute (ignored by Angular):

```
<!-- ❌ Angular ignores this in reactive forms — shows warn
ing in console -->
<input [disabled]="true" formControlName="name"/>
```

The FormControl is still **enabled** — both `form.value` and `getRawValue()` include it.

Right way 1 — disable in TypeScript after init:

```
ngOnInit() {
  this.form.controls.name.disable()  // ✅
}

// re-enable later
this.form.controls.name.enable()
```

Right way 2 — disable at initialization:

```
name: new FormControl({ value: '', disabled: true }, { nonN
ullable: true })
//                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//                      1st arg: { value, disabled } object
//                      2nd arg: options object
```

Rule — two separate objects:

Object	Keys allowed
1st argument	`value` + `disabled` only
2nd argument	`nonNullable` + `validators` + `asyncValidators`

Effect on form.value vs getRawValue():

Control state	`form.value`	`form.getRawValue()`
Enabled	included	included
Disabled	**excluded**	**included**

> Rule in Reactive Forms: never control state via HTML — always do it in TypeScript.

10. Providing Values — setValue vs patchValue

Setting each control manually (old verbose way):

```
this.form.controls.name.setValue(res.name)
this.form.controls.author.setValue(res.author)
this.form.controls.category.setValue(res.category)
// ... repeat for every field
```

`setValue()` — all or nothing:

```
this.form.setValue(res)
```

Must provide **every single control** — no missing fields:

```
// ✅ all fields provided
this.form.setValue({
  id: 1, name: 'Book', author: 'Asif',
  publishedYear: '2024', rating: 5,
  publisher: { publisherName: 'ABC', publisherType: 'tradit
ional' },
  category: 'Programming', description: '...'
})

// ❌ throws error — missing fields
this.form.setValue({ name: 'Book' })
```

`patchValue()` — partial update (preferred):

```
this.form.patchValue(res)           // full object ✅
this.form.patchValue({ name: 'X' }) // partial ✅
```

Missing fields are simply ignored — no error.

Comparison:

	`setValue()`	`patchValue()`
All fields required	✅ yes — throws if missing	❌ no — missing ignored
Partial update	❌ no	✅ yes
Best for	full object load	API response or partial update

> Professional choice: use `patchValue(res)` when loading from API — cleaner and safer.

---

11. Submitting and Saving Form Data

Template wiring:

```
<form [formGroup]="form" (ngSubmit)="saveBook()">
  <button type="submit">Save</button>
</form>
```

The null problem with `Partial<Book>`:

```
// service expects:
save(book: Partial<Book>)

// Partial<Book> expands to:
{ name?: string; author?: string; ... }
// string | undefined — undefined is OK, null is NOT
```

---

12. Nested FormGroups

Defining nested group in TypeScript:

```
form = this.formBuilder.nonNullable.group({
  name: [''],
  publisher: this.formBuilder.nonNullable.group({  // ← nes
ted group
    publisherName: [''],
    publisherType: ['']
```

```
  })
})
```

> **Important:** `nonNullable` does NOT automatically apply to nested groups. You must add `nonNullable` to each nested group separately.

Wiring nested group in template:

```
<section formGroupName="publisher">  <!-- connects to neste
d FormGroup -->
  <input formControlName="publisherName"/>
  <input formControlName="publisherType"/>
</section>
```

Accessing nested controls:

```
// dot notation (typed FormGroup)
this.form.controls.publisher.controls.publisherName.setValu
e('ABC')
```

---

16. Interview Questions

Reactive Forms vs Template Driven Forms

Q1. What is the difference between Reactive Forms and Template Driven Forms?

> Reactive Forms are defined in TypeScript — the form structure, validators, and state are all controlled in the component class. Template Driven Forms are defined in the HTML using `ngModel`. Reactive Forms give full type safety, better testability, and are preferred for complex forms. Template Driven Forms are simpler but `form.value` is typed as `any` — no compile-time safety.

Q2. Why does `form.value` return `any` in Template Driven Forms but a typed object in Reactive Forms?

> In Template Driven Forms, Angular builds the form dynamically at runtime from the template — TypeScript cannot see the controls at compile time, so it

> types `form.value` as `any`. In Reactive Forms, you define controls in TypeScript — so TypeScript knows exactly what fields exist and what types they hold, giving a fully typed `form.value`.

FormControl & nonNullable

Q3. Why does `FormControl` always include `null` in its type?

> Because calling `.reset()` on a FormControl sets the value to `null` by default. TypeScript reflects this — if reset can produce `null`, the type must include `null`. That is why `new FormControl('Asif')` gives `FormControl<string | null>` and not `FormControl<string>`.

Q4. What does `nonNullable: true` do in a FormControl?

> It changes the reset behavior — instead of resetting to `null`, the control resets to its initial value. TypeScript then removes `null` from the type. So `new FormControl('', { nonNullable: true })` gives `FormControl<string>` instead of `FormControl<string | null>`.

Q5. What is the difference between `form.value` and `form.getRawValue()`?

> `form.value` excludes disabled controls — `form.getRawValue()` includes disabled controls. Also.

FormBuilder

Q6. Does `nonNullable` on the outer `FormBuilder.nonNullable.group()` apply to nested groups automatically?

> No. `nonNullable` only applies to the group it is called on. Nested `FormGroup` instances must each have `nonNullable` applied separately:

```
form = this.fb.nonNullable.group({
  publisher: this.fb.nonNullable.group({ ... })  // must re
peat nonNullable here
})
```

---

setValue vs patchValue

Q7. What is the difference between `setValue()` and `patchValue()`?

> `setValue()` requires all controls to be provided — it throws an error if any field is missing. `patchValue()` accepts a partial object — missing fields are ignored. For loading API data,